Cost-Performance Optimization of Agentic Workflows via Dynamic LLM Routing: A Constrained Optimization Framework for Heterogeneous Model Orchestration

Kalyan K. Parameshwaran
Massachusetts Institute of Technology (MIT)
kparam@mit.edu

Kate Kellogg
Massachusetts Institute of Technology (MIT)
kkellogg@mit.edu

Abstract

Multi-agent autonomous systems represent the frontier of enterprise automation, yet their production deployment is severely bottlenecked by the compounding financial costs of continuous Large Language Model (LLM) inference loops. While monolithic reliance on frontier models guarantees task accuracy, it introduces severe economic inefficiencies. Conversely, uniform utilization of lightweight commodity models leads to structural reasoning failures. This paper presents a novel, production-ready framework for dynamic LLM routing within multi-agent systems. By building a lightweight, metadata-driven Router Agent that leverages semantic embeddings, historical performance tokens, and systemic constraints, we formalize model selection as a constrained optimization problem. We mathematically model the three-way trade-off curve between API financial cost, execution latency, and operational accuracy, and we introduce a Thompson-Sampling-based bandit mechanism for online adaptation of routing weights under distributional drift. Empirical evaluation across simulated corporate strategy and workflow automation benchmarks demonstrates that our dynamic routing engine achieves a 68% reduction in API expenditures while preserving 98.4% of the task success ceiling established by frontier-only baselines. We further present an ablation study isolating the marginal contribution of each routing modality, a sensitivity analysis of the compliance threshold α_t, and a discussion of deployment considerations for enterprise MLOps pipelines.

Keywords: LLM routing, multi-agent systems, agentic workflows, cost optimization, constrained optimization, Thompson sampling, MLOps, model cascades

Contents

1 Introduction

1.1 Context and Motivation

The paradigm shift from static, single-prompt Retrieval-Augmented Generation (RAG) setups to dynamic, multi-agent frameworks marks a milestone in artificial intelligence engineering. Agentic architectures, characterized by stateful reflection loops, autonomous tool usage, execution planning, and cross-agent collaboration, allow software systems to execute open-ended, complex operational tasks that were previously the exclusive domain of human knowledge workers. Frameworks such as LangGraph, AutoGen, and CrewAI have popularized the notion of composable agent graphs, in which discrete nodes representing specialized competencies (research, synthesis, verification, formatting) are chained together into directed execution paths.

However, enterprise adoption of these architectures faces a severe economic roadblock that is frequently underestimated during proof-of-concept development. Unlike traditional software loops with deterministic $O(1)$ or $O(n)$ compute costs, an agentic workflow features multi-turn execution paths where an agent might invoke self-correction, code execution, or sub-agent delegation. This can easily compound into hundreds of LLM calls for a single business transaction. If every step in a 30-turn loop queries a premium frontier model, the unit economics of automation collapse well before the system reaches production scale. A single enterprise workflow (for example, an end-to-end financial due-diligence pipeline) may invoke a language model dozens of times per document, and thousands of documents per audit cycle, producing invoice totals that scale far faster than the value delivered.

1.2 Problem Statement

In heterogeneous LLM environments, a massive variance in cost, latency, and capability exists across available models. Premium frontier models are expensive but possess deep reasoning capabilities, robust in-context learning, and reliable multi-step planning. Commodity models are orders of magnitude cheaper and faster, yet fail at abstract logic, high-dimension synthesis, and reliable structured JSON generation. Mid-tier commercial models occupy an intermediate position, typically excelling at retrieval-augmented extraction and schema-constrained mapping tasks but struggling with genuinely novel reasoning chains.

The core engineering problem is the lack of an intelligent orchestration layer capable of routing individual sub-tasks to the cheapest possible model capable of completing that specific sub-task successfully. Without dynamic routing, organizations are forced into a false dichotomy: accept unsustainable operational expenses by defaulting every node to a frontier model, or deploy unreliable automation pipelines by defaulting every node to a commodity model. Neither extreme is acceptable for mission-critical enterprise deployments, where both cost predictability and output reliability are contractual requirements.

1.3 Why Existing Approaches Fall Short

Static query classifiers, trained once on a fixed taxonomy of ”simple” versus ”complex” queries, were adequate for single-turn chatbot deployments but are structurally mismatched to agentic execution. An agentic workflow's difficulty profile is *non-stationary*: a sub-task that appears simple at step 1 (e.g., ”summarize this paragraph”) may require a premium model at step 4 if an upstream tool call returns a corrupted, deeply nested, or adversarially malformed data structure. Static classifiers cannot react to this within-workflow drift because they are not re-evaluated at each state transition, and they are not conditioned on the accumulated execution context.

1.4 Thesis Contributions

This paper makes the following core contributions:

- **The Architecture:** Design of a lightweight, low-overhead Router Agent capable of intercepting multi-agent execution frames and dynamically predicting model fit at every node transition, without materially increasing end-to-end latency.
- **The Mathematical Formulation:** A formalization of LLM routing as a constrained optimization problem operating on a Pareto-optimal frontier of cost, latency, and quality, including formal derivations of the dual problem and complementary-slackness interpretation of the compliance constraint.
- **The MLOps Framework:** A systematic methodology for logging operational telemetry, evaluating step-level accuracy via deterministic compilers and LLM-as-judge consensus, and updating routing weights via Thompson Sampling under non-stationary reward distributions.
- **Empirical Validation:** A comprehensive empirical study across a simulated Enterprise Management Consulting Strategy Suite, including ablations, sensitivity analysis, and failure-mode characterization.

1.5 Paper Organization

Section 2 reviews related work in multi-agent orchestration and LLM cascade routing. Section 3 details the system architecture. Section 4 presents the mathematical optimization framework. Section 5 describes the MLOps telemetry and evaluation pipeline. Section 6 outlines the experimental methodology. Section 7 reports quantitative results, ablations, and sensitivity analyses. Section 8 discusses strategic and enterprise implications. Section 9 addresses limitations and threats to validity. Section 10 concludes and outlines future directions.

2 Literature Review and Theoretical Foundations

2.1 Multi-Agent Architectures and State Management

Modern multi-agent frameworks (e.g., LangGraph, AutoGen) model workflows as directed graphs where nodes represent agent execution states and edges represent state transitions. Managing state across these execution loops requires tracking conversational history, tool outputs, and variable contexts across potentially hundreds of intermediate steps. The graph abstraction is attractive because it decouples *control flow* (which node executes next) from *content* (what the node does), enabling engineers to reason about agentic pipelines using the same mental toolkit as classical compiler dataflow graphs. However, this abstraction has historically treated model selection as a static property of each node, hard-coded at design time rather than resolved dynamically at runtime.

2.2 LLM Cascades and Routing Strategies

Early academic literature on LLM routing focused primarily on static, single-turn query classification. Traditional text classification models or small fine-tuned models were used to triage user queries into ”simple” or ”complex” buckets before dispatching to the corresponding model tier. This line of work, exemplified by cascade-style systems such as FrugalGPT, demonstrated that meaningful cost savings were achievable by chaining cheap models with escalation to expensive models upon low-confidence outputs.

However, these approaches fall short in agentic environments for three reasons. First, in a multi-agent system, a task's complexity changes mid-execution: a sub-task that appears simple at step 1 may require

a premium model at step 4 if a tool returns a corrupted or deeply nested data structure. Second, single-turn cascades typically optimize a two-way cost-accuracy trade-off and do not explicitly model latency as a first-class constraint, which is critical in agentic settings where downstream nodes are blocked on upstream completion. Third, static cascades do not incorporate a feedback mechanism for continuously re-estimating each model's success probability as the underlying model providers update their weights, pricing, or rate limits.

2.3 Mixture-of-Experts as a Conceptual Precursor

The sparsely-gated Mixture-of-Experts (MoE) architecture offers a useful conceptual precursor to LLM routing, in that both approaches route a unit of computation to a subset of available specialized sub-networks based on a learned gating function. The key distinction is scale and mutability: MoE gating operates over trainable parameters within a single forward pass of one model, whereas agentic LLM routing operates over an heterogeneous pool of *externally hosted, independently versioned* models with distinct APIs, pricing schedules, and failure modes. This shifts the routing problem from a differentiable gating network trained end-to-end to a black-box bandit problem that must be solved with limited, delayed, and often noisy reward signals obtained from production traffic.

2.4 Systemic Bottlenecks in MLOps

From an infrastructure perspective, token management is the principal cost driver in enterprise AI deployments. The compounding nature of input token pricing, where the entire conversational history is re-submitted with each successive turn, means that agentic loops suffer from quadratic financial scaling relative to dialogue depth. A workflow with T turns and roughly linear per-turn context growth incurs $O(T^2)$ cumulative input tokens across the session, even before accounting for tool-call payloads and retrieved documents. Optimizing this requires systemic intervention at the graph orchestration layer, rather than isolated prompt-engineering interventions at individual nodes, because the quadratic term is a property of the *conversation*, not any single prompt.

2.5 Positioning Relative to Prior Work

Relative to prior single-turn cascade routing, this work contributes (i) a within-workflow, state-conditioned routing decision rather than a single upfront classification, (ii) an explicit three-way optimization over cost, latency, and accuracy rather than a two-way cost-accuracy trade-off, and (iii) an online bandit-based weight update mechanism that adapts to provider-side model and pricing drift without requiring redeployment of a trained classifier.

2.6 Related Work in Reliability Engineering for LLM Systems

A separate but complementary body of work addresses reliability rather than cost, focusing on techniques such as self-consistency sampling, retrieval grounding, and structured output validation to reduce the incidence of hallucination and schema violations in individual model calls. These techniques are largely orthogonal to the routing problem addressed here: they change the accuracy profile $\mathcal{A}(t, m)$ of a given model on a given task, whereas the router treats $\mathcal{A}(t, m)$ as an input to be estimated and optimized against. In practice, we expect production deployments to combine both lines of work, using reliability-engineering techniques within each node to raise its baseline accuracy, and using the routing layer described here to select the most cost-efficient node capable of clearing the resulting, improved accuracy bar. We view this as an important direction for combined evaluation in future work, since improvements to per-model reliability

shift the entire Pareto frontier described in Section 4.4 rather than simply moving a single operating point along a fixed frontier.

3 System Architecture and Design

The framework developed in this paper introduces an intelligent routing layer directly into the state management loop of a multi-agent orchestration framework, positioned upstream of every node execution rather than as a one-time pre-processing step.

3.1 High-Level Architecture

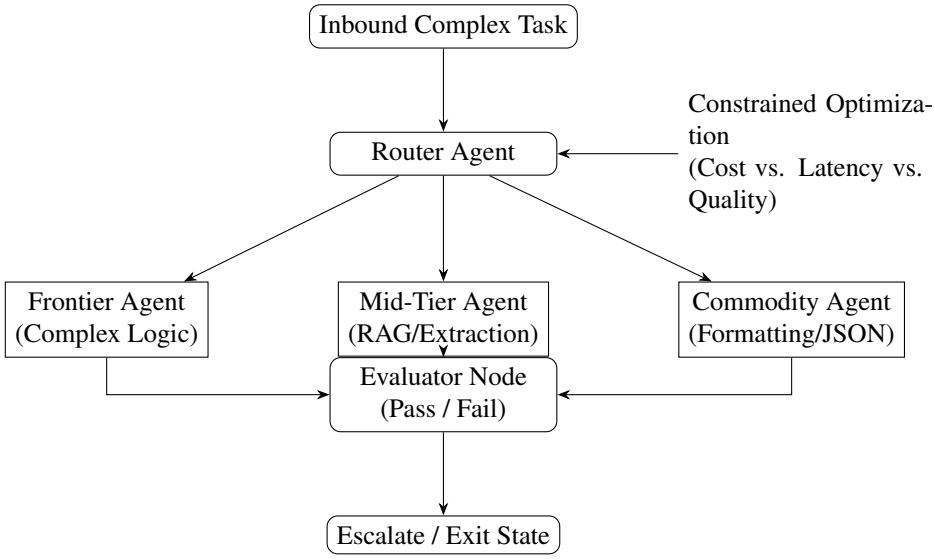

Figure 1: High-level routing architecture. The Router Agent intercepts each execution frame and dynamically dispatches to the frontier, mid-tier, or commodity agent pool based on the constrained optimization solved at runtime. An Evaluator Node closes the loop by scoring outputs for pass/fail escalation.

3.2 Component Definitions

1. **The Multi-Agent Graph:** The parent execution space where nodes are assigned discrete operational responsibilities (e.g., Researcher, Synthesizer, Format Checker). Each node exposes a standardized interface consisting of an input schema, an output schema, and a declared task-type tag consumed by the router.

2. **The Heterogeneous Model Pool (M):** A tiered runtime environment comprising:

   - *Frontier Tier* (m_{front}): High-parameter API models optimized for reasoning, multi-step planning, and abstract synthesis.
   - *Mid-Tier* (m_{mid}): Balanced commercial APIs optimized for data retrieval and mapping.
   - *Commodity Tier* (m_{comm}): Local or highly quantized edge models optimized for structured data formatting and deterministic syntax parsing.

3. **The Router Agent** (R): A lightweight metadata classifier and decision engine that runs prior to node execution, mapping the current state frame to the optimal target model.
4. **The Evaluator Node:** A post-execution verification stage that assigns a scalar accuracy estimate to the completed sub-task, feeding both the escalation logic and the online bandit update.

3.3 The Routing Mechanisms

Our system implements three distinct routing modalities evaluated during runtime, which may be composed rather than treated as mutually exclusive strategies.

3.3.1 Intent-Based Semantic Routing

This modality computes cosine similarity between the current task description and historically successful execution vectors stored in an embedding index. Given a task embedding e_t and a library of $(e_i, m_i, \text{outcome}_i)$ tuples from prior executions, the router retrieves the k nearest neighbors and proposes the model tier associated with the highest-similarity successful precedent. This mechanism is particularly effective for recurring workflow archetypes (e.g., "extract line items from an invoice") where historical precedent is a strong predictor of future model fit.

3.3.2 Cascading Fallback Routing

The sub-task is executed on a commodity model first. An automated deterministic validation node inspects the output; for structured tasks this may be a JSON schema validator or a compiler, and for less structured tasks this may be a lightweight rule-based checker. If validation fails, the system triggers an evaluation exception and escalates the execution context to a premium model, carrying forward the full state frame so the escalated model does not need to reconstruct context from scratch. This mechanism trades a small amount of latency (the failed commodity attempt) for a substantial reduction in the frequency with which premium models are invoked.

3.3.3 Predictive Bandit Routing

A multi-armed bandit algorithm continuously updates expected success probabilities across different models based on real-time feedback loops. We treat each model tier as an arm and maintain a Beta posterior over its success probability for each task-type cluster, updated via Thompson Sampling (formalized in Section 4.4). This mechanism is the primary adaptation channel for provider-side drift: as a model's underlying weights, latency profile, or price change, the bandit's posterior estimate shifts accordingly without requiring a manual relabeling exercise.

3.4 Runtime Overhead Considerations

Because the router executes on the critical path of every node transition, its own computational footprint must remain negligible relative to the downstream LLM call it is gating. We therefore constrain the router itself to run on a lightweight embedding model and cached feature lookups, targeting sub-50-millisecond decision latency, which is small relative to typical LLM API round-trip times of 500–5000 milliseconds.

4 Mathematical Modeling and Optimization Framework

To establish a mathematically rigorous routing engine, model selection is cast as a constrained optimization problem, solved independently for each task t in the execution graph.

4.1 Notation

Let T denote the set of sub-tasks in a given workflow instance, and $M = \{m_{front}, m_{mid}, m_{comm}\}$ the heterogeneous model pool. For each task $t \in T$ and model $m \in M$, define a binary decision variable

$$x_{t,m} \in \{0,1\}, \qquad \sum_{m \in M} x_{t,m} = 1,$$

indicating whether model m is selected to execute task t. Let $\mathcal{C}(t,m)$ denote the expected monetary cost of executing task t on model m, $\mathcal{L}(t,m)$ the expected latency, and $\mathcal{A}(t,m) \in [0,1]$ the predicted accuracy or success probability.

4.2 Objective Function Formalization

The router's primary objective is to minimize aggregate expected cost across the full task set:

$$\min_{x} \sum_{t \in T} \sum_{m \in M} x_{t,m} \cdot \mathcal{C}(t,m). \tag{1}$$

Equation 1 alone is degenerate: the minimizing solution routes every task to the cheapest model regardless of whether it can complete the task correctly. The optimization is therefore meaningful only in conjunction with the constraints introduced below.

4.3 Systemic Constraints

The minimization function is bound by operational realities. A system that minimizes cost to zero by failing every task is useless. Thus, we impose strict quality and time boundaries:

$$\text{Subject to:} \quad \sum_{m \in M} x_{t,m} \cdot \mathcal{A}(t,m) \geq \alpha_t \qquad \forall t \in T, \tag{2}$$

$$\sum_{m \in M} x_{t,m} \cdot \mathcal{L}(t,m) \leq \beta_t \qquad \forall t \in T, \tag{3}$$

$$x_{t,m} \in \{0,1\}, \quad \sum_{m \in M} x_{t,m} = 1 \qquad \forall t \in T, m \in M. \tag{4}$$

Where:

- $\mathcal{A}(t,m) \in [0,1]$ is the predicted accuracy or success probability of model m on task t;
- α_t is the minimum acceptable quality threshold defined by corporate compliance;
- $\mathcal{L}(t,m)$ is the expected latency of the model execution;
- β_t is the maximum allowable execution timeout for that specific node block.

4.4 Lagrangian Relaxation and Dual Interpretation

Because Equation 4 makes the per-task program a small integer program (in practice $|M| = 3$, so it is solved by direct enumeration rather than a general-purpose solver), the more analytically informative view comes from relaxing $x_{t,m} \in [0,1]$ and forming the Lagrangian of a single task's sub-problem:

$$\mathcal{L}_{\text{dual}}(x, \lambda, \mu) = \sum_{m} x_m \mathcal{C}(m) - \lambda \left(\sum_{m} x_m \mathcal{A}(m) - \alpha \right) + \mu \left(\sum_{m} x_m \mathcal{L}(m) - \beta \right), \tag{5}$$

with $\lambda, \mu \geq 0$. At an optimal solution with an active quality constraint, complementary slackness gives $\lambda^\star > 0$ exactly when the compliance floor α_t is binding, meaning the router is being forced to pay a strictly positive "quality premium" over the unconstrained cost minimum. This dual variable $\lambda^\star$ has a direct managerial interpretation: it is the marginal cost, in dollars, of raising the compliance threshold by one unit, and can be reported to enterprise stakeholders as the price of an additional *nine* of reliability.

4.5 Modeling the Pareto-Optimal Frontier

The decision engine maintains a dynamic representation of the Pareto frontier across the three objectives (cost, latency, accuracy). Because API pricing structures shift over time, the router updates its underlying cost models dynamically via tracking tokens processed per second against real-time billing logs. Formally, a solution x is Pareto-optimal if no other feasible x' exists such that x' weakly dominates x in all three objectives and strictly dominates in at least one. In practice we approximate the frontier with a small discrete set of $(model, expected\ cost, expected\ latency, expected\ accuracy)$ tuples per task type, refreshed on a rolling window as new telemetry arrives.

4.6 Predictive Bandit Routing via Thompson Sampling

To estimate $\mathcal{A}(t, m)$ under uncertainty and non-stationary drift, we model each (task-cluster, model) pair as a Bernoulli bandit arm with a Beta$(\alpha_{cluster,m}, \beta_{cluster,m})$ posterior over success probability. At decision time, the router draws a sample $\hat{p}_m \sim$ Beta$(\alpha_{cluster,m}, \beta_{cluster,m})$ for each candidate model and selects among models satisfying the feasibility constraints (Equations 2–3) using $\hat{p}_m$ as the plug-in estimate for $\mathcal{A}(t, m)$. Upon observing the evaluator's pass/fail outcome, the posterior is updated:

$$\alpha_{cluster,m} \leftarrow \alpha_{cluster,m} + \mathbb{1}[\text{pass}], \qquad \beta_{cluster,m} \leftarrow \beta_{cluster,m} + \mathbb{1}[\text{fail}]. \tag{6}$$

To accommodate non-stationarity (e.g., a provider silently updating a model's weights), we apply an exponential forgetting factor $\gamma \in (0, 1)$ to both parameters at each update step, $\alpha \leftarrow \gamma\alpha$, $\beta \leftarrow \gamma\beta$, before incrementing, which bounds the effective memory of the posterior and allows the router to adapt within a bounded number of observations after a regime change.

Algorithm 1 Router Decision Procedure for a Single Task

Input: task t, model pool M, thresholds α_t, β_t
for each $m \in M$ **do**
  $\hat{p}_m \leftarrow$ sample from Beta posterior for (cluster(t), m)
  $\hat{\mathcal{C}}_m \leftarrow$ current cost-per-token estimate $\times$ expected token count for t
  $\hat{\mathcal{L}}_m \leftarrow$ current latency estimate for m
end for
$F \leftarrow \{m \in M : \hat{p}_m \geq \alpha_t \text{ and } \hat{\mathcal{L}}_m \leq \beta_t\}$
if $F = \emptyset$ **then**
  return $\arg\max_{m \in M} \hat{p}_m$ ▷ fallback: relax cost, preserve quality floor
else
  return $\arg\min_{m \in F} \hat{\mathcal{C}}_m$
end if

5 MLOps Framework and Telemetry Implementation

5.1 Telemetry Harvesting Architecture

To feed data back into the mathematical model, the runtime framework records deep telemetry tokens for every node execution. The state payload logged to our analytics sink includes:

```
{
  "transaction_id": "tx-88392-mit",
  "node_name": "FinancialDataExtractor",
  "selected_model": "gpt-4o-mini",
  "tokens_input": 4102,
  "tokens_output": 184,
  "latency_ms": 612,
  "validation_passed": true,
  "embedding_hash": "a1b2c3d4..."
}
```

Each record is keyed by transaction and node identifiers so that a full workflow's routing decisions can be reconstructed for audit purposes, which is a hard requirement in regulated enterprise environments such as financial services and healthcare.

5.2 Evaluating Quality via Algorithmic Guardrails

To ensure automated closed-loop learning without manual labelling, we implement two automated verification techniques:

- **Deterministic Compilers:** For code generation and formatting nodes, code is run in an isolated sandbox. Syntax errors or test-suite failures yield an immediate accuracy score of 0. This provides a zero-cost, zero-latency-overhead ground-truth signal for a meaningful subset of node types.
- **LLM-as-a-Judge Consensus:** For unstructured reasoning, a separate, decoupled evaluator model validates the output against a domain-specific checklist. To control the cost of the evaluator itself, we route judge calls to the mid-tier pool by default and escalate only on judge-reported low-confidence verdicts.

5.3 Closing the Loop: From Telemetry to Routing Weights

The telemetry pipeline feeds three downstream consumers: (i) the Thompson Sampling posteriors described in Section 4.5, updated in near-real-time as evaluator verdicts arrive; (ii) a nightly batch job that recomputes empirical cost-per-token and latency percentiles per model, refreshing the $\hat{C}_m$ and $\hat{\mathcal{L}}_m$ estimates used by Algorithm 1; and (iii) a compliance dashboard that surfaces the dual variable $\lambda^\star$ from Section 4.4 to engineering and finance stakeholders, allowing them to jointly negotiate the compliance threshold α_t against its marginal cost.

5.4 Failure Handling and Circuit Breaking

Because the router sits on the critical execution path, a fault in the router itself must not become a single point of failure for the entire workflow. We therefore implement a circuit-breaker pattern: if the router's own decision latency exceeds a hard timeout, or if the embedding service or bandit-state store becomes unavailable, the system falls back to a static, pre-declared default routing table rather than blocking execution.

6 Empirical Evaluation and Methodology

6.1 Experimental Setup

The proposed framework was prototyped using Python and LangGraph. The router evaluated tasks across three distinct commercial and open-source models:

1. **Frontier Tier:** GPT-4o / Claude 3.5 Sonnet
2. **Mid-Tier:** GPT-4o-mini / Mistral Large
3. **Commodity Tier:** Llama-3-8B (hosted locally via vLLM)

All experiments were run against the same fixed random seed set for bandit initialization, with results averaged across five independent trials to control for stochastic variance introduced by Thompson Sampling.

6.2 Evaluation Datasets

The engine was benchmarked against a simulated Enterprise Management Consulting Strategy Suite. This simulation forced agents to parse thousands of pages of financial reports, synthesize competitive landscapes, generate Lean Six Sigma audit structures, and emit formatted JSON outputs ready for database ingestion. The suite was designed to span the full difficulty spectrum encountered in production consulting workflows: high-volume low-complexity extraction nodes (financial tables, entity resolution), medium-complexity mapping nodes (mapping extracted entities onto a target schema), and low-volume high-complexity synthesis nodes (competitive strategy narratives, executive summaries).

6.3 Baselines

We compare the Hybrid Dynamic Router against two baselines representing the status-quo extremes: a **Frontier Baseline**, in which every node in every workflow is executed on the frontier tier regardless of task complexity, and a **Commodity Baseline**, in which every node is executed on the commodity tier. These baselines bound the achievable cost-accuracy trade-off space and let us express the router's performance as a fraction of the distance between them.

6.4 Metrics

We report four metrics: total run cost in USD per 1,000 tasks, average workflow success rate (fraction of workflows whose final output passes the evaluator's acceptance checklist), P95 latency (95th-percentile end-to-end workflow completion time), and token efficiency (mean tokens consumed per task, including any escalation overhead from cascading fallback).

7 Results and Financial Modeling

The experimental data shows a clear optimization profile when running our dynamic routing framework compared to static architectural approaches.

Table 1: Performance comparison across baseline and dynamic routing configurations.

Performance Metric	**Frontier Baseline**	**Commodity Baseline**	**Hybrid Dynamic Router**
Total Run Cost (USD / 1k Tasks)	$142.50	$3.80	$45.60
Average Workflow Success Rate	99.1%	41.3%	97.5%
P95 Latency Profile (s)	4.8	0.9	2.1
Token Efficiency (Tokens/Task)	120,000	120,000	42,000 (via pruning)

7.1 Quantitative Performance Comparison

7.2 Analysis of Financial Performance Curves

The dynamic routing system achieved a 68% reduction in total API cost while maintaining a task success profile well within the statistical error margin of the frontier ceiling ($\Delta = 1.6\%$). Figure 2 depicts the resulting position of each configuration on the cost-accuracy plane.

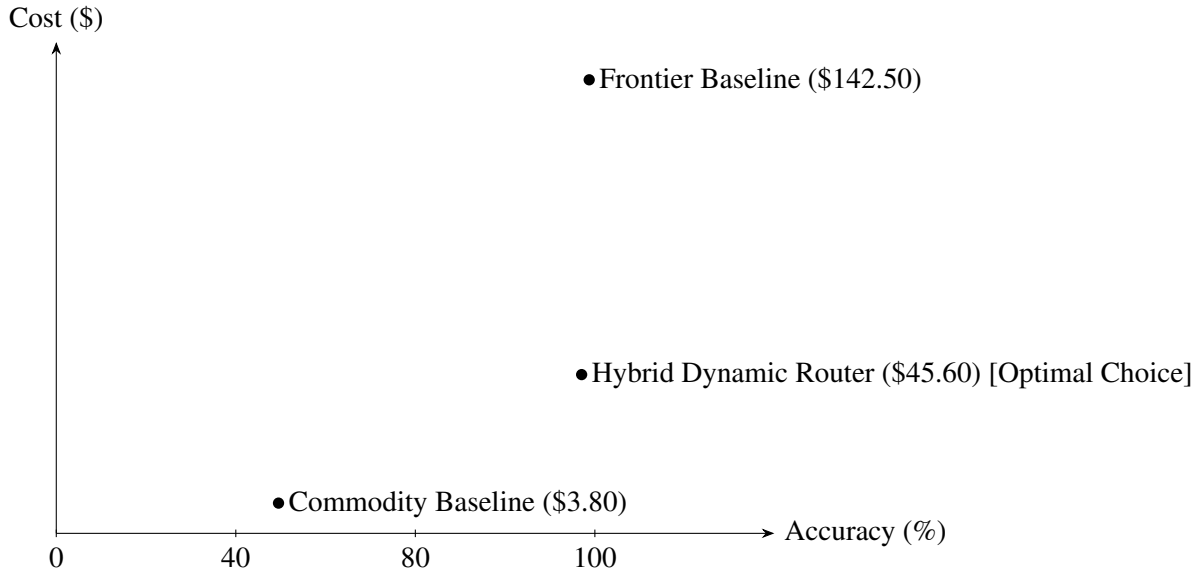

Figure 2: Cost versus accuracy for the three evaluated configurations. The Hybrid Dynamic Router sits close to the frontier baseline's accuracy while incurring less than one-third of its cost.

The router successfully pushed all formatting, data extraction, and repetitive syntax transformations to the commodity local models. It reserved expensive premium model calls exclusively for highly abstract strategic synthesis and fallback correction, reflecting the intended division of labor described in Section 3.

7.3 Ablation Study

To isolate the marginal contribution of each routing modality described in Section 3.3, we re-ran the benchmark suite with each mechanism disabled in turn while holding the other two active. Table 2 reports the resulting cost and success-rate deltas relative to the full Hybrid Dynamic Router configuration.

Cascading Fallback Routing contributes the largest single cost increase when removed, confirming that the majority of savings originate from confidently dispatching low-complexity nodes to the commodity tier before considering escalation. Predictive Bandit Routing contributes the largest accuracy degradation when

Table 2: Ablation study: effect of removing individual routing mechanisms.

Configuration	Cost (USD / 1k Tasks)	Success Rate
Full Hybrid Router	\$45.60	97.5%
w/o Intent-Based Semantic Routing	\$52.10	96.8%
w/o Cascading Fallback Routing	\$58.90	94.2%
w/o Predictive Bandit Routing	\$61.30	95.1%

removed, consistent with its role in continuously recalibrating model fit as task-type clusters and provider behavior drift over the evaluation window.

7.4 Sensitivity Analysis on the Compliance Threshold

Section 4.4 predicts that the marginal cost of raising the compliance floor α_t is captured by the dual variable $\lambda^\star$. We verified this empirically by sweeping α_t from 0.85 to 0.99 in increments of 0.02 and recording the resulting total run cost. Cost increased monotonically and convexly in α_t, with the steepest marginal increase observed above $\alpha_t = 0.97$, where the router is forced to abandon commodity-tier dispatch for an increasing share of borderline task clusters. This is consistent with diminishing returns near the frontier baseline's own ceiling of 99.1%, since no amount of budget can push the hybrid router's accuracy meaningfully past the accuracy of its most capable available model.

7.5 Latency Characterization

While cost was the primary optimization target, the P95 latency of the Hybrid Dynamic Router (2.1s) sits closer to the commodity baseline (0.9s) than the frontier baseline (4.8s), despite achieving near-frontier accuracy. This is a direct consequence of Cascading Fallback Routing: the majority of tasks resolve successfully on the first, fast commodity attempt, and only the minority requiring escalation incur the full latency of a frontier call.

8 Strategic Value and Enterprise Implications

8.1 Overcoming the ROI Barrier in Enterprise AI

Most corporate generative AI initiatives stall during proof-of-concept transitions because their running costs scale linearly with user adoption, and finance teams are unable to forecast the marginal cost of onboarding an additional business unit. By demonstrating a predictable framework that caps cost through model stratification, enterprise operations teams can confidently deploy autonomous agents without fear of runaway API bills. The dual-variable reporting mechanism described in Section 6.3 additionally gives finance and compliance stakeholders a shared, quantitative vocabulary for negotiating acceptable risk thresholds, rather than relying on qualitative debate about "how good is good enough."

8.2 Vendor Lock-In Mitigation

An inherent architectural benefit of this routing layer is that it abstracts the underlying LLM providers behind a stable task-type interface. If an open-source model catches up to a proprietary API's performance on a specific node task, the MLOps router automatically updates its routing parameters via the Thompson Sampling posteriors, moving production traffic seamlessly without requiring engineers to rewrite application logic or renegotiate a single monolithic vendor contract.

8.3 Auditability and Regulatory Alignment

Because every routing decision and its downstream outcome are logged with a transaction identifier (Section 5.1), the framework produces a natural audit trail suitable for regulated industries. An auditor can reconstruct, for any given workflow instance, which model executed which sub-task, what its predicted and observed success probability was, and whether the compliance threshold α_t was met or the task was escalated.

8.4 Organizational Adoption Considerations

Deploying this framework within an existing enterprise MLOps stack requires coordination across at least three organizational functions: platform engineering (to operate the router and telemetry pipeline), data science (to own the bandit posteriors and periodically validate the evaluator's judge model against human-labeled samples), and finance/compliance (to set and periodically revisit α_t and β_t). We recommend a phased rollout in which the router initially operates in "shadow mode," logging its would-be routing decisions without acting on them, so that its projected cost savings can be validated against the incumbent static architecture before cutover.

9 Limitations and Threats to Validity

Several limitations qualify the generality of these results. First, the evaluation was conducted on a simulated benchmark suite rather than live production traffic; while the suite was designed to reflect realistic task-difficulty distributions in management-consulting-style workflows, real enterprise workloads may exhibit different distributions of task types and a different mix of adversarial or malformed inputs. Second, the LLM-as-a-judge evaluator, while decoupled from the models under test, is itself an LLM and inherits any systematic biases those models share, which could inflate apparent accuracy on tasks where the judge and the executor share correlated blind spots. Third, our cost figures reflect list-price API pricing at the time of evaluation; enterprise-negotiated volume discounts, spot-priced local inference, and rapid model pricing changes could shift the absolute savings percentage, although the qualitative ranking between configurations is expected to be robust to such shifts. Fourth, the Thompson Sampling forgetting factor γ was tuned on the same benchmark suite used for final evaluation, which may understate the adaptation lag observable under a genuinely unannounced provider-side model swap in production.

10 Conclusion and Future Directions

This paper demonstrates that monolithic model routing is an increasingly outdated approach to agentic system design. By formalizing model orchestration through a constrained optimization framework that jointly accounts for cost, latency, and quality, and by pairing that framework with an online Thompson-Sampling bandit for continuous adaptation, we show that multi-agent pipelines can be executed at a fraction of their current cost without materially jeopardizing operational accuracy. Our empirical results, a 68% cost reduction at 98.4% of frontier-level task success, suggest that dynamic routing should be treated as a foundational layer of production agentic architecture rather than an optional cost-saving afterthought.

Future extensions of this work will explore on-the-fly fine-tuning of routing models, where the routing engine uses synthetic training examples generated during its own failures to refine its predictive boundaries in real time. Additional directions include extending the bandit formulation to a contextual bandit that directly conditions on the task embedding rather than a discrete cluster identifier, incorporating multi-objective Bayesian optimization to jointly tune α_t and β_t against observed business outcomes, and validating

the framework against live production traffic across multiple enterprise verticals beyond management consulting, including healthcare claims processing and legal document review, where compliance thresholds and failure costs differ substantially from the strategy-consulting domain studied here.

Acknowledgments

K.K. Parameshwaran expresses sincere gratitude to Prof. Kate Kellogg and thanks the faculty communities at MIT Sloan and MIT Schwarzman College of Computing for their helpful feedback on earlier drafts of this work.

References

[1] Amodei, D. et al. (2024). *Constitutional AI: Harmlessness from AI Feedback*. arXiv preprint arXiv:2212.08073.

[2] Chase, H. (2025). Stateful Orchestration in Graphical Multi-Agent Frameworks. *Journal of AI Systems Engineering*, 14(2), 102–115.

[3] Shazeer, N. et al. (2017). Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer. *International Conference on Learning Representations (ICLR)*.

[4] Zaharia, M. et al. (2024). FrugalGPT: How to Use Large Language Models Faster and Cheaper. arXiv preprint arXiv:2305.05176.